
Nhắc lại kiến thức bài trước

Giới thiệu về lập trình hướng đối tượng



Sự phát triển của ngôn ngữ lập trình

Ngôn ngữ máy

Lập trình
tuần tự

Hướng
thủ tục

Hướng
đối tượng

Lập trình tuần tự

- Mã nguồn được viết dưới dạng "đầu tiên làm thế này, sau đó làm thế kia"
- Khó sửa lỗi, bảo trì.

```
RELOAD EQU 0E6H    ; defining reload constant for baudrate generation
ORG 0000H           ; org directive
SJMP START          ; jump to main program

SENDCH:  ORG 0023H   ; ISR for RI/TI interrupt
          CLR TI      ; clear transmit flag
          MOV SBUF, #'A' ; send the ASCII value of 'A'
          RETI        ; return back to main program

START:    ANL PCON, #7FH ; Set SMOD=0
          ANL TMOD, #0FH ; Alter only the setting of Timer-1
          ORL TMOD, #20H ; Timer-1 in mode-2
          MOV TH1, #RELOAD ; Move the reload value to TH1
          SETB TR1      ; start Timer-1 for baud rate generation
          MOV SCON, #40H ; set serial port in mode-1
          ORL IE, #90H  ; Enable serial port interrupt

          MOV SBUF, #'A' ; Transmit a character

WAIT:    SJMP WAIT    ; wait till interrupt occurs

END
```

Sự phát triển của ngôn ngữ lập trình

Ngôn ngữ máy

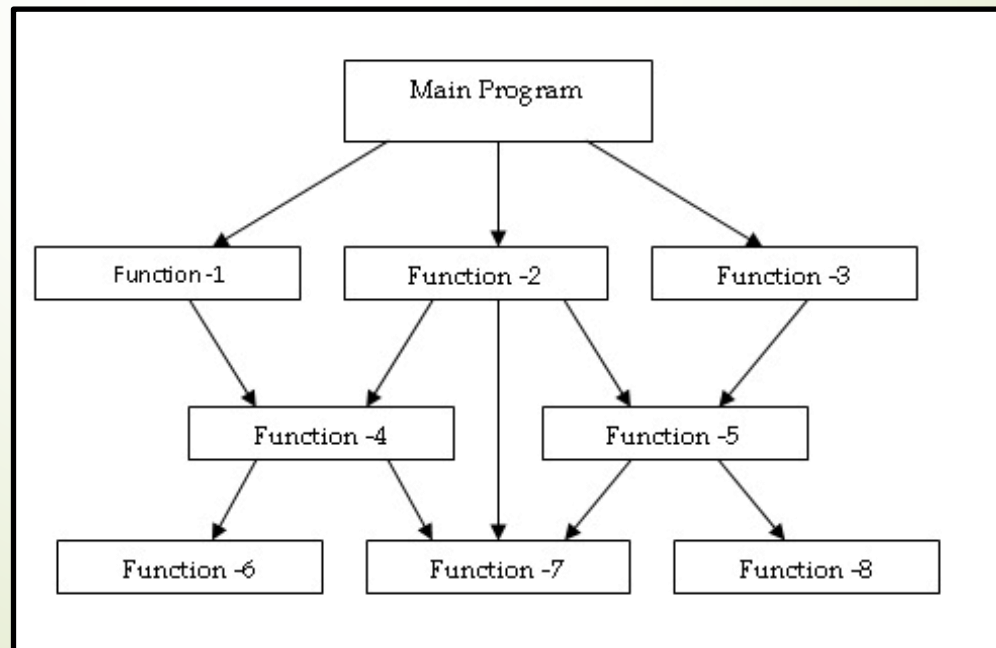
Lập trình
tuần tự

Hướng
thủ tục

Hướng
đối tượng

Lập trình hướng thủ tục (Lập trình cấu trúc)

- Xây dựng chương trình dựa trên các hàm/thủ tục/chương trình con
- Dữ liệu và xử lý (hàm) tách rời nhau
- Các hàm không bắt buộc phải tuân theo một cách thức chung truy cập vào dữ liệu



Đối tượng

Đối tượng là gì?

"Tất cả mọi thứ đều là đối tượng"

-Alan Kay





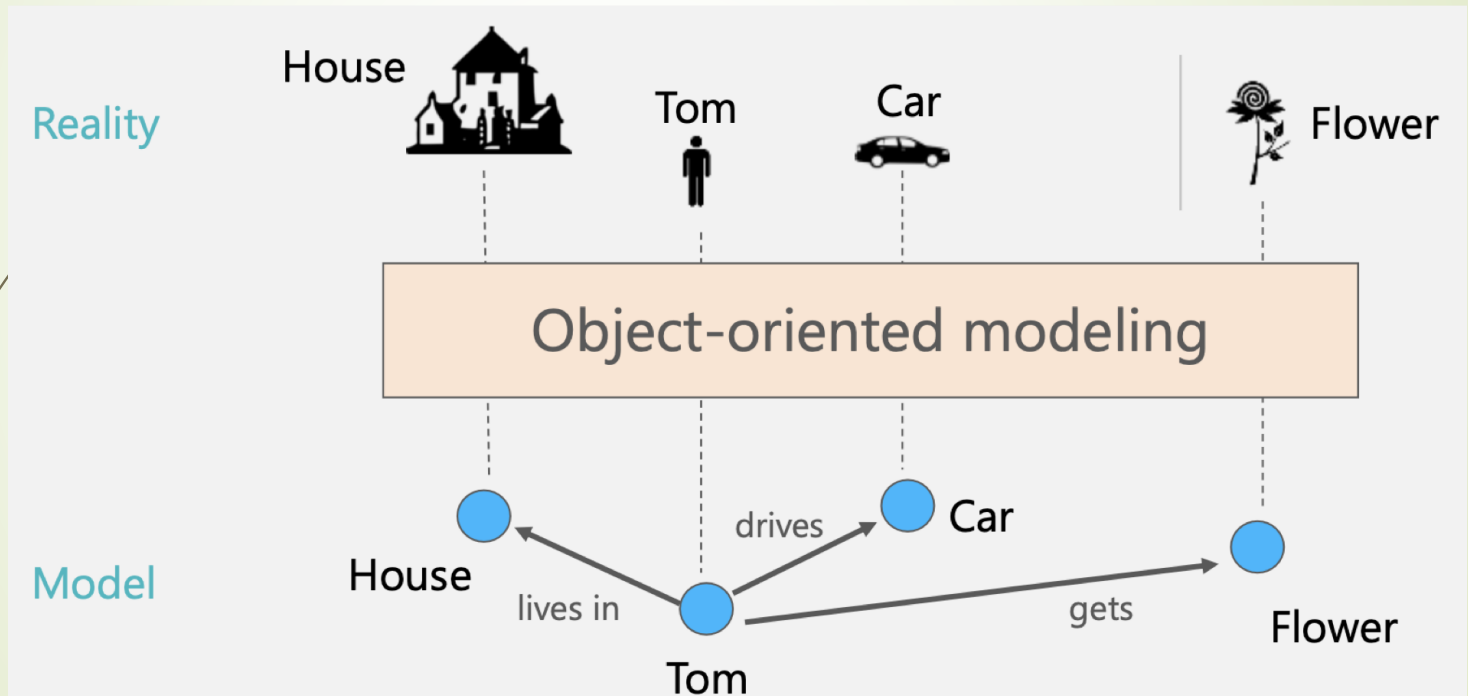
In an interview on education in America with the Davis Group Ltd., Kay said:

I had the misfortune or the fortune to learn how to read fluently starting about the age of three, so I had read maybe 150 books by the time I hit first grade, and I already knew the teachers were lying to me. [\[ref\]](#)

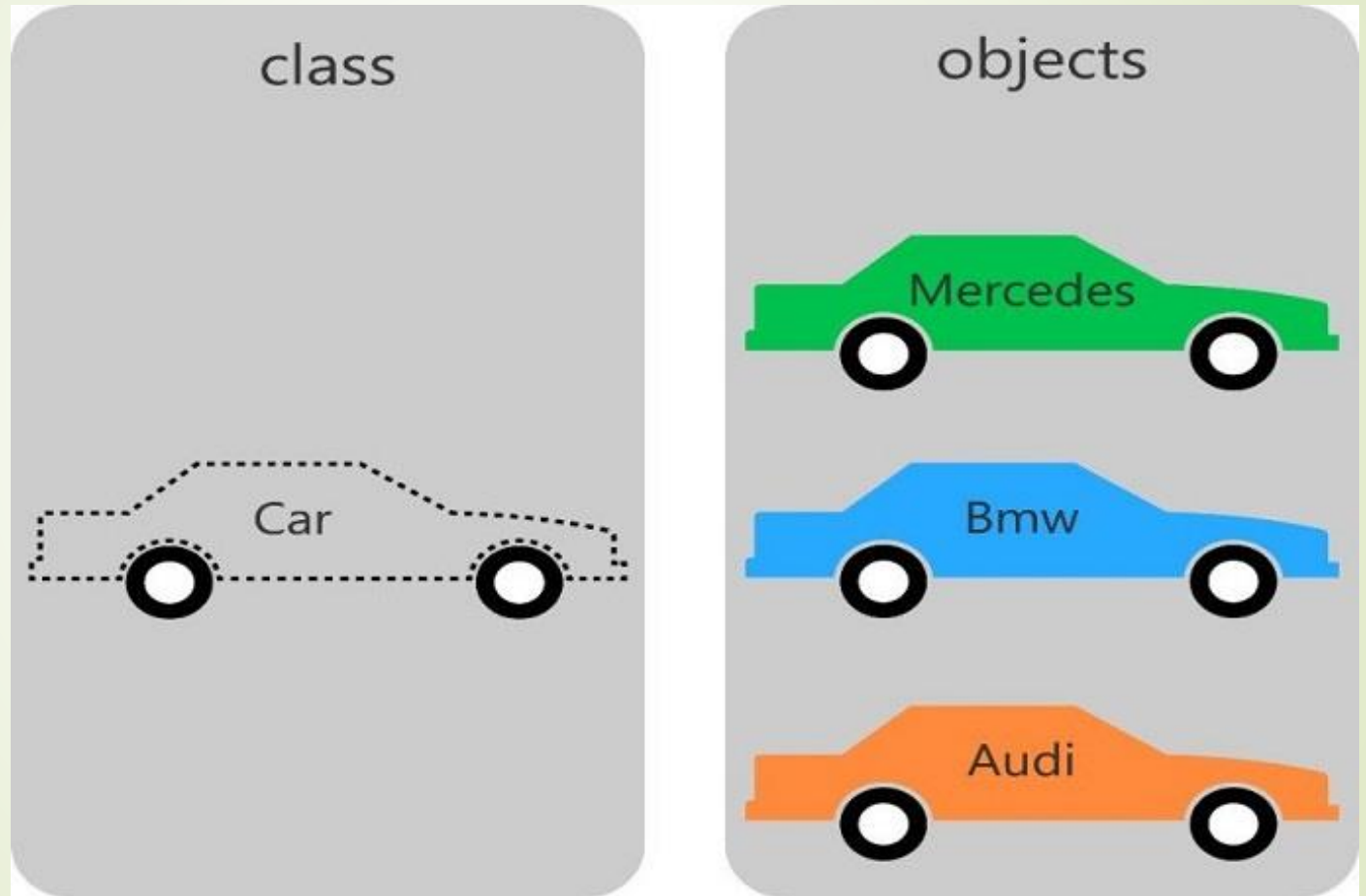
Tôi đã không may hoặc may mắn được học cách đọc thông thạo bắt đầu từ khoảng ba tuổi, vì vậy tôi đã đọc khoảng 150 cuốn sách khi tôi vào lớp một, và tôi đã biết rằng các giáo viên đã nói dối tôi.

Lập trình hướng đối tượng

Coi chương trình phần mềm là một tập hợp các đối tượng tương tác với nhau

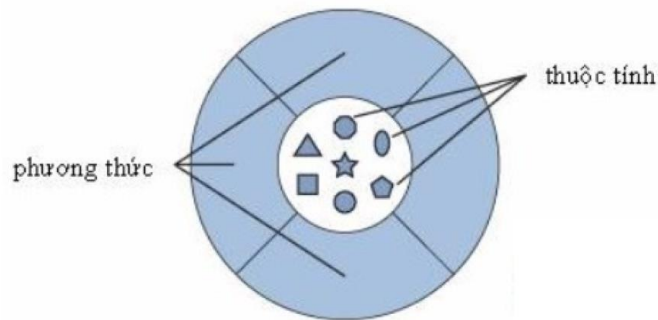


Lớp và đối tượng

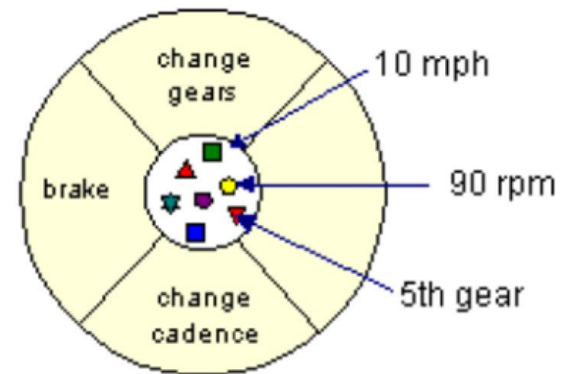


Đối tượng phần mềm

- Đối tượng (object) là một thực thể phần mềm bao bọc các **thuộc tính** và các **phương thức** liên quan.
 - Thuộc tính được xác định bởi giá trị cụ thể gọi là **thuộc tính thể hiện**
 - Một đối tượng cụ thể được gọi là một **thể hiện**.



Đối tượng phần mềm



Đối tượng phần mềm **Xe Đạp**

Các nguyên lý cơ bản

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

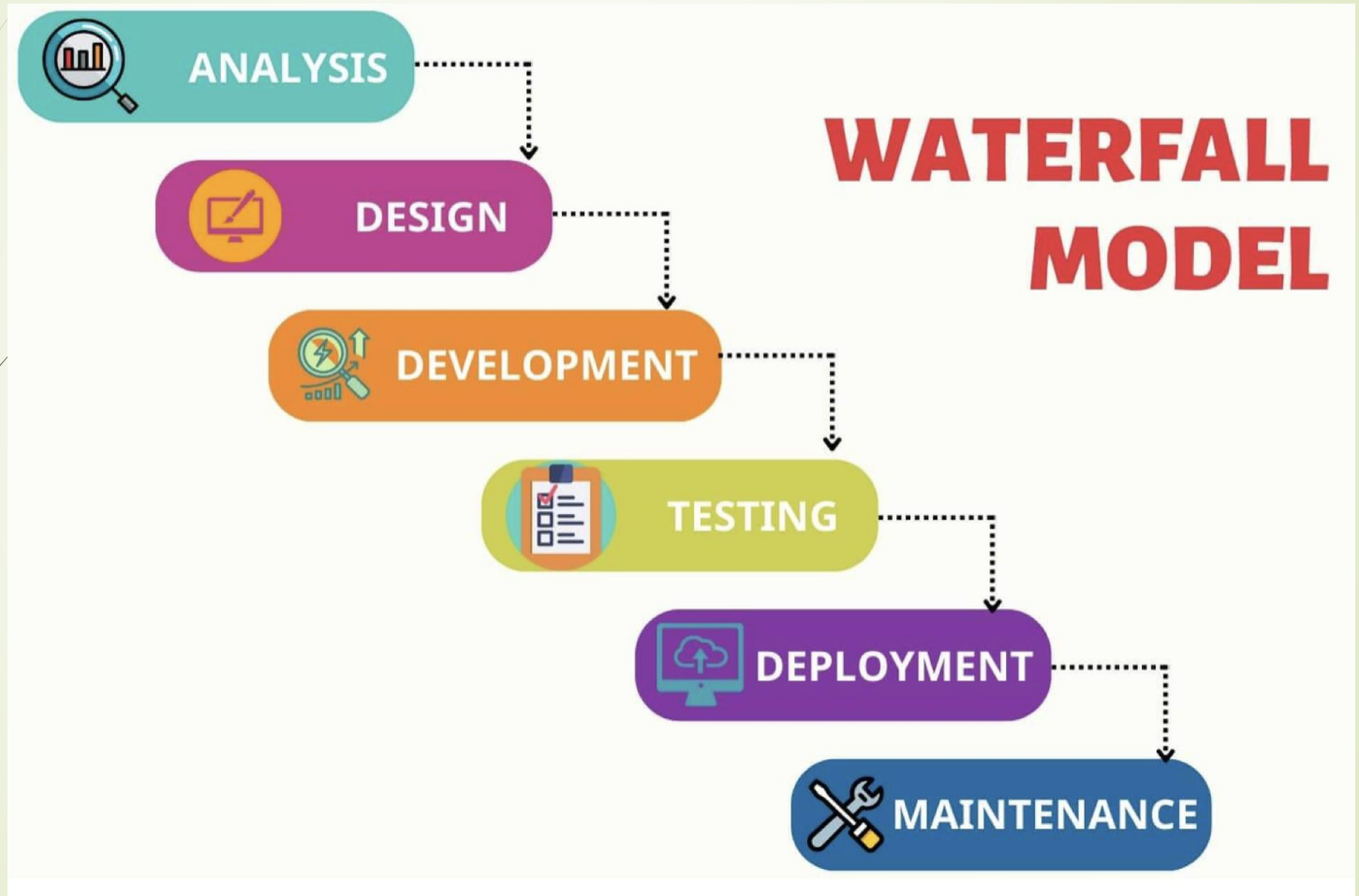
*Trừu tượng
hóa*

Đóng gói

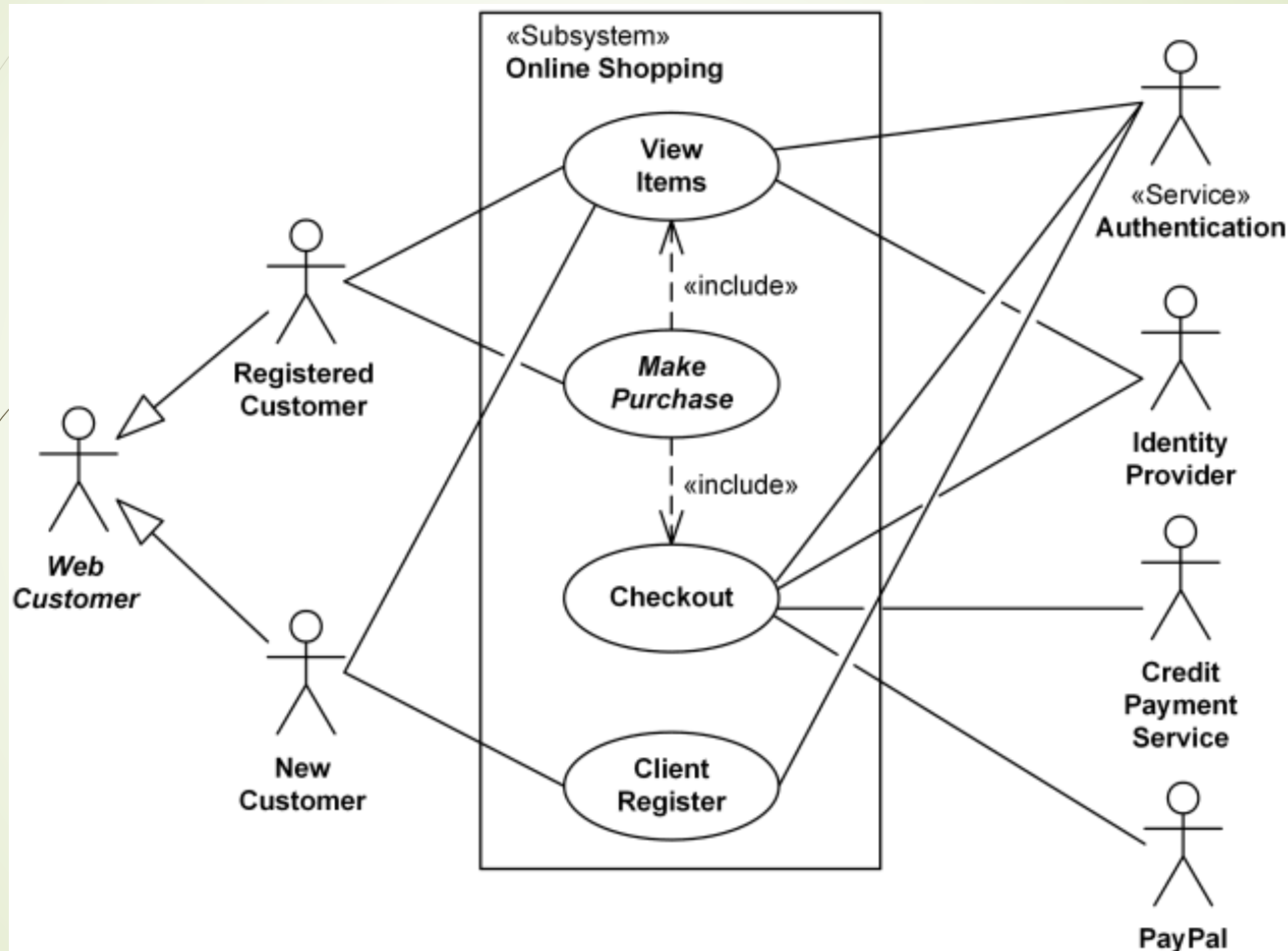
*Module
hóa*

Phân cấp

Phát triển phần mềm

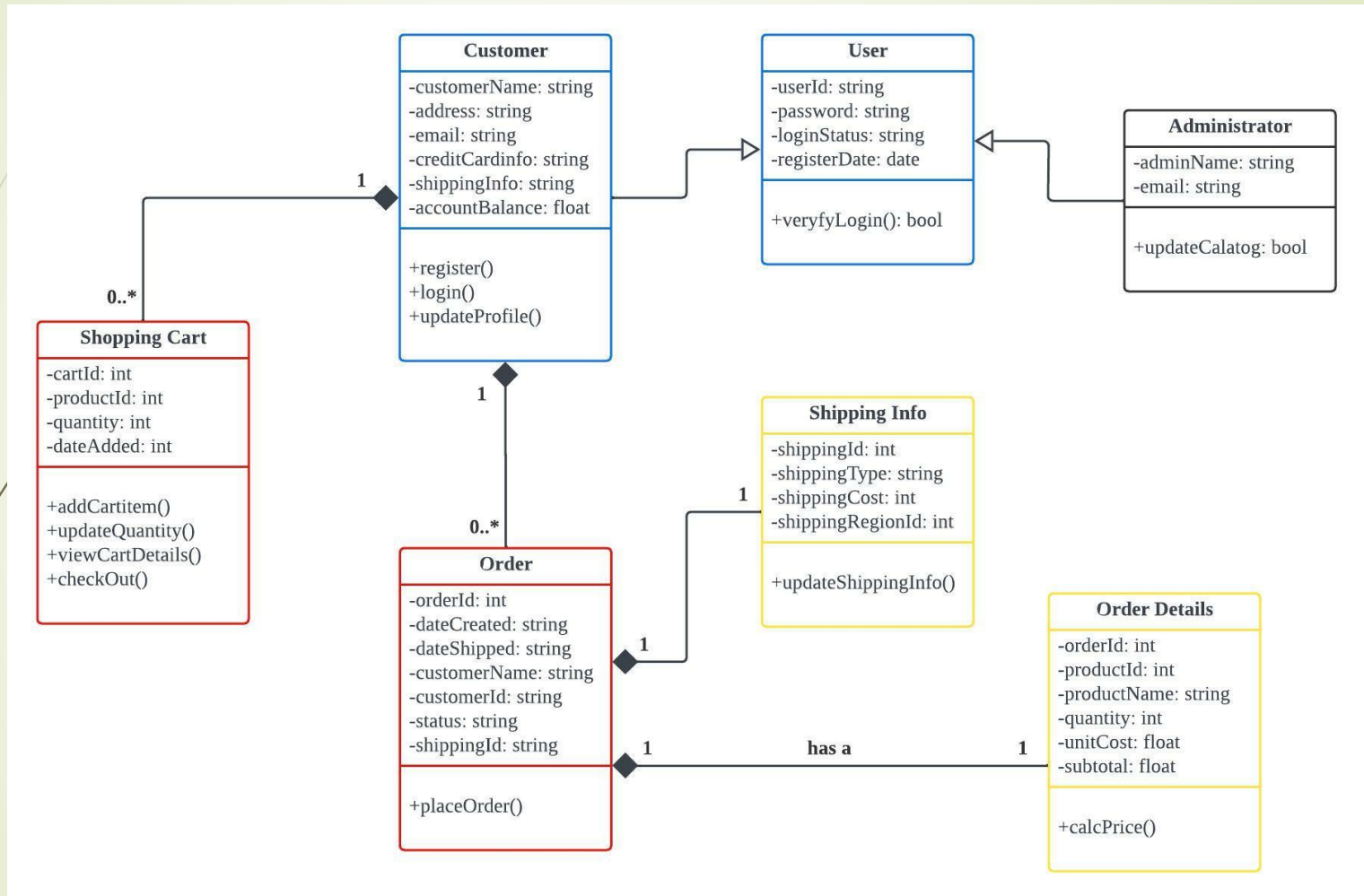


Use Case Diagram (Biểu đồ Use case)

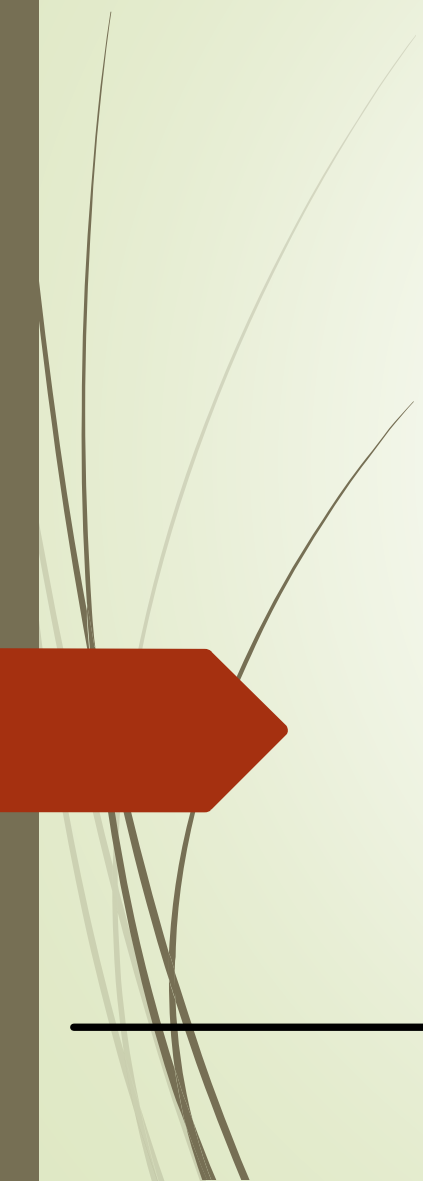


Sơ đồ Use Case của một shop bán hàng online

Class Diagram (Biểu đồ lớp)

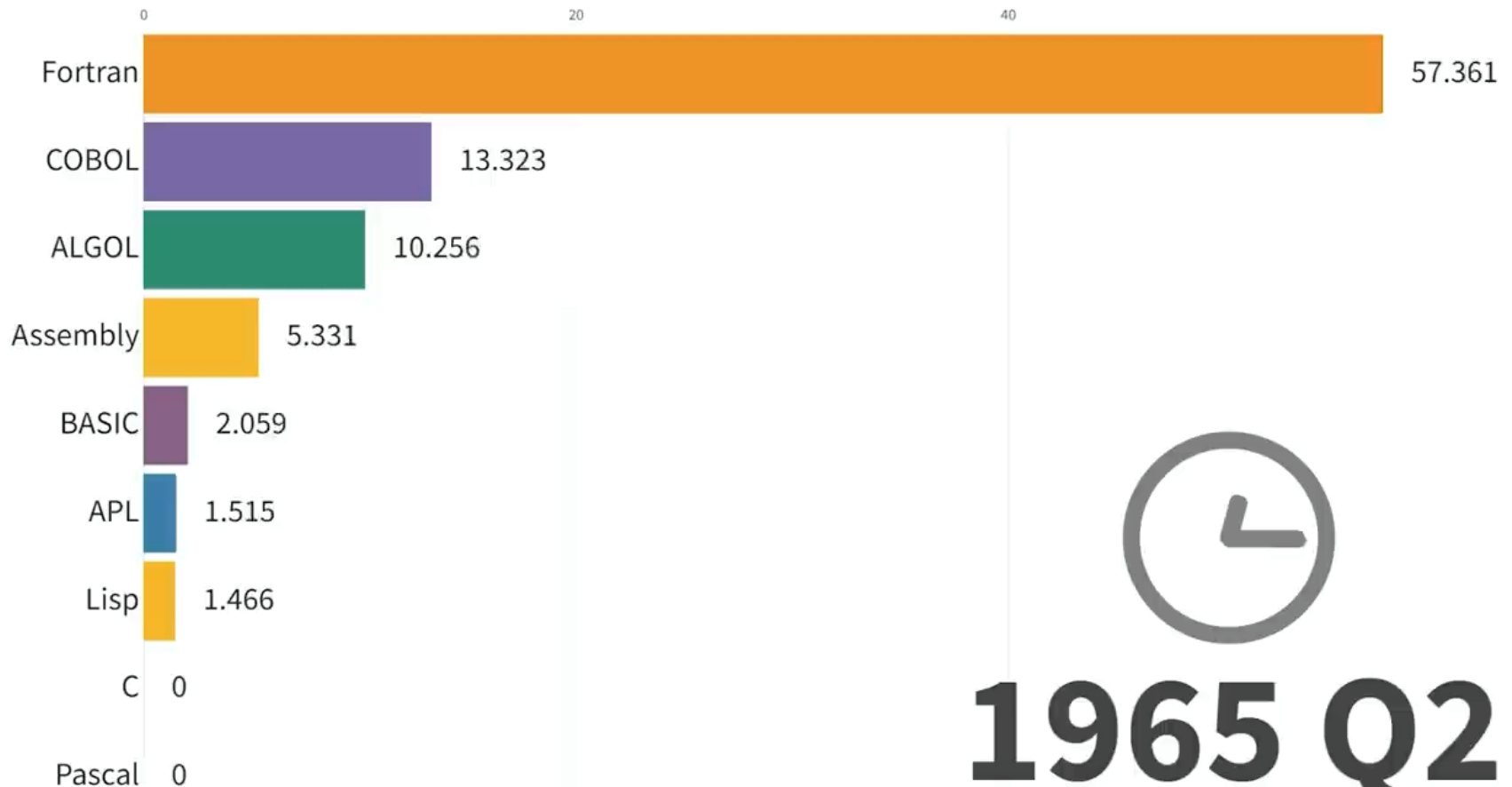


Sơ đồ lớp của một shop bán hàng online



OOP02: Python cơ bản

MOST POPULAR PROGRAMMING LANGUAGE (1965-2022)



Một số ngôn ngữ OOP phổ biến nhất



Một số ngôn ngữ OOP phổ biến nhất

Nên học ngôn ngữ lập trình nào?

Em muốn theo hướng web thì nên học gì?

Em muốn dev game thì học gì?

Theo AI/Data Analysis thì lộ trình ntn?

Cơ hội việc làm sau khi ra trường của ngành CNTT?

CNTT gồm những mảng nào ạ?

Trường nào dạy CNTT tốt?

Em học đến năm thứ 2 mà nản quá, nhờ mọi người tư vấn?



[What Programming Language Should I Learn First in 2024?](#)
[\[Solved\] \(freecodecamp.org\)](#)

Tham khảo khác: [1](#), [2](#), [3](#)

Nội dung

1. Giới thiệu Python
2. Định danh
3. Lập trình căn bản



1

Giới thiệu Python

*Python là ngôn ngữ lập trình cấp cao,
tương tác, thông dịch, hướng đối tượng*

Giới thiệu

- Python là ngôn ngữ lập trình cấp cao, tương tác, thông dịch, hướng đối tượng
- Do Guido van Rossum đề xuất từ 1985 – 1990
- Mã nguồn Python: giấy phép GNU General Public License (GPL)
- Python sẵn dùng trên Unix, Linux, Mac OS, Windows
- Top 5 ngôn ngữ lập trình phổ biến
- Được sử dụng bởi các tổ chức tập đoàn như Wikipedia, Google, Yahoo!, CERN, NASA, Facebook, Amazon, Instagram, Spotify

Giới thiệu

- Dễ học (Easy-to-learn)
- Dễ đọc (Easy-to-read)
- Dễ bảo trì (Easy-to-maintain)
- Thư viện chuẩn (standard library): tương thích UNIX, Linux, Windows và Mac
- Chế độ tương tác: thực thi, kiểm tra, gỡ rối
- Khả chuyển (Portable): phần cứng, hệ điều hành
- Khả năng mở rộng (Extendable): thêm mô-đun
- Kết nối với các hệ quản trị CSDL
- Lập trình giao diện đồ họa GUI
- Quy mô chương trình (Scalable)

Giới thiệu

- Python được sử dụng trong giảng dạy,
- Tính toán khoa học,
- Công nghệ sinh-tin học,
- Phát triển ứng dụng Web,
- Lập trình ứng dụng mạng, nghiên cứu an ninh mạng,
- Kỹ thuật đồ họa, xử lý ảnh và thị giác máy tính,
- Máy học và khai thác dữ liệu, xử lý ngôn ngữ tự nhiên, trí tuệ nhân tạo
- Lập trình nhúng,
- Quản trị hệ thống,
- Lập trình trò chơi, v.v.



Triết lý căn bản của ngôn ngữ Python được trình bày trong tài liệu The Zen of Python (PEP 20), có dạng thơ Haiku, tóm gọn như sau:

- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Complex is better than complicated.
- Readability counts.

- Đẹp dễ tốt hơn xấu xí
- Minh bạch tốt hơn ngầm định
- Đơn giản tốt hơn phức tạp
- Phức tạp tốt hơn rắc rối
- Tính dễ đọc rất quan trọng.

PYTHON LIBRARIES & FRAMEWORKS

MACHINE LEARNING:



WEB DEVELOPMENT:



AUTOMATION TESTING:

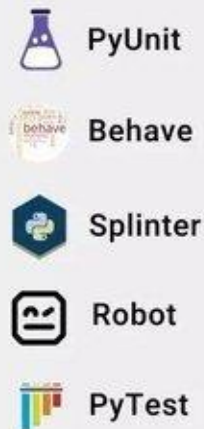


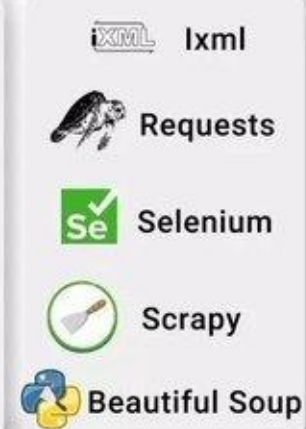
IMAGE PROCESSING:

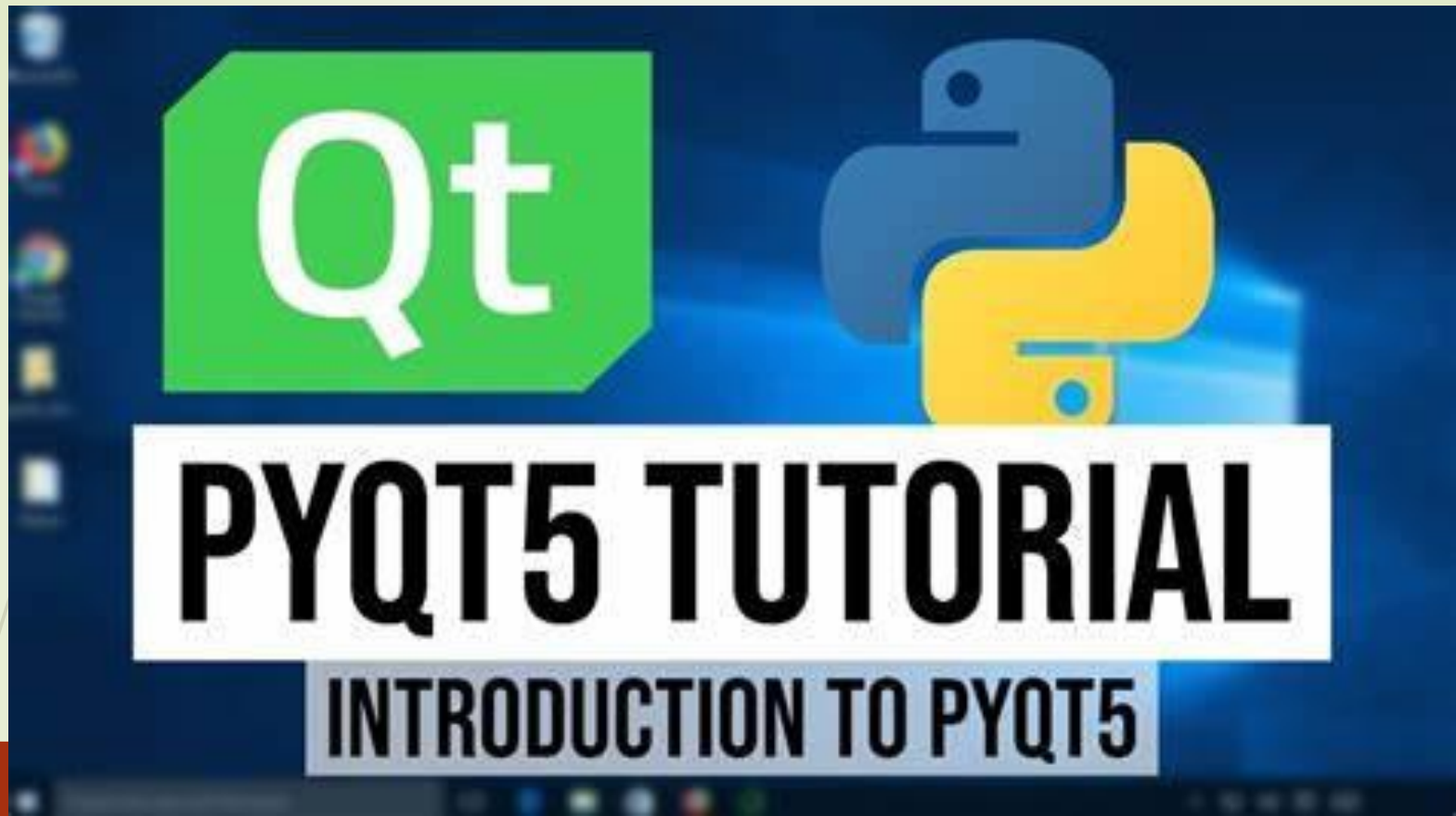


GAME DEVELOPMENT:

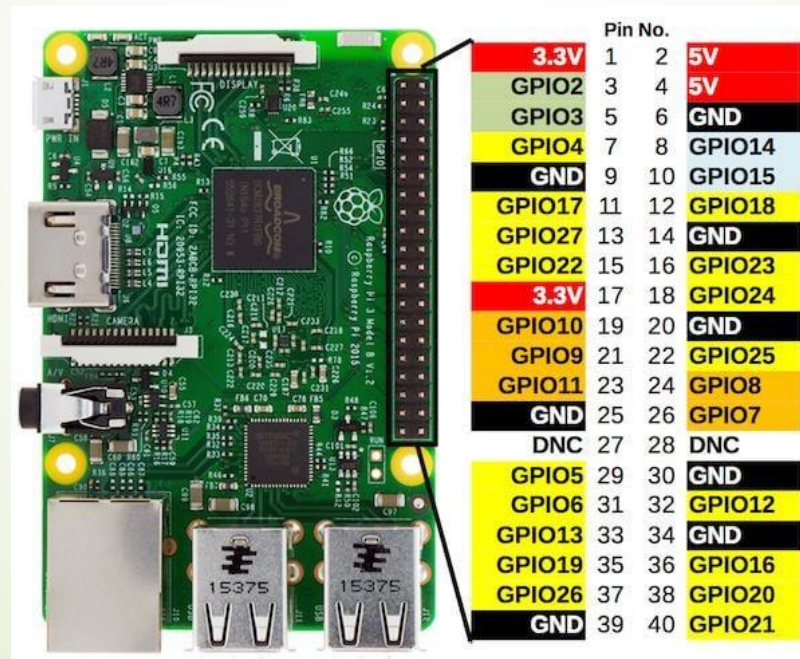


WEB SCRAPING:





GUI programming with Python 2



GPIO pins with Python
(Embedded systems and IoT, Edge Computing)

2



Định danh (Identifier)

Xâu ký tự thể hiện tên các biến, các phương thức, các lớp và nhãn

Định danh

- Định danh:
 - Xâu ký tự thể hiện tên các biến, các phương thức, các lớp và nhãn
- Quy định với định danh:
 - Các ký tự có thể là chữ số, chữ cái, '\$' hoặc '_'
 - Tên không được phép:
 - Bắt đầu bởi một chữ số
 - Trùng với từ khóa
 - Phân biệt chữ hoa chữ thường
 - Yourname, yourname, YourName và yourName là 4 định danh khác nhau



An_Identifier
a_2nd_Identifier
Go2
\$10



An-Identifier
2nd_Identifier
assert
10\$

Quy ước đặt tên

- Quy ước với định danh (naming convention [3](#)):
 - Bắt đầu bằng chữ cái
 - Gói (package): tất cả sử dụng chữ thường
`theexample`
 - Lớp (Class): viết hoa chữ cái đầu tiên trong các từ ghép lại
`TheExample`
 - Phương thức/thuộc tính (method/attribute): Bắt đầu bằng chữ thường, viết hoa chữ cái đầu tiên trong các từ còn lại
`theExample`
 - Hằng (constants): Tất cả viết hoa
`THE_EXAMPLE`

Các từ khóa

Keywords:

```
[help> keywords
```

```
Here is a list of the Python keywords.  Enter any keyword to get more help.
```

False	break	for	not
None	class	from	or
True	continue	global	pass
__peg_parser__	def	if	raise
and	del	import	return
as	elif	in	try
assert	else	is	while
async	except	lambda	with
await	finally	nonlocal	yield

3

Lập trình căn bản

Trình thông dịch, chương trình đầu tiên, cú pháp, lệnh print, lệnh input, biến, kiểu cơ bản, phép toán, cấu trúc điều khiển (rẽ nhánh, lặp), hàm, các kiểu dữ liệu phức

Trình thông dịch Python

```
[nghi@localhost ~]$ python
Python 2.7.5 (default, Apr 10 2015, 08:09:05)
[GCC 4.8.3 20140911 (Red Hat 4.8.3-7)] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> print 'Hello World!'
Hello World!
>>> █
```

Soạn thảo chương trình: PyCharm, Geany, notepad++, Nano, Gedit, **Visual Studio Code**, Sublime text, Jupiter notebook (Anaconda Navigator),... v.v.

Chương trình **hello.py**

```
#!/usr/bin/python  
print 'Hello World!'
```

Thực thi **hello.py**

```
[nghi@localhost bg]$ python hello.py  
Hello World!
```

- Phân biệt ký tự thường hoa
- Các từ khóa của Python

and	exec	not
as	finally	or
assert	for	pass
break	from	print
class	global	raise
continue	if	return
def	import	try
del	in	while
elif	is	with
else	lambda	yield
except		

- Sử dụng **#** để chú thích 1 dòng trong chương trình

```
# comment
```

- Sử dụng **'''** để chú thích 1 đoạn

```
'''
```

```
print("We are in a comment")  
print ("We are still in a comment")  
'''
```

- Dấu **'** và **"**

```
word = 'word'  
sentence = "This is a sentence."
```

```
message = """This message will  
... span several lines."""
```

Chú ý:

"straight" and
"curly" quotes

- Sử dụng canh lề (bắt buộc) để bao các khối lệnh của hàm, lớp hoặc luồng điều khiển
- Số khoảng trắng dùng để canh lề có thể nhiều ít tùy ý nhưng tất cả lệnh trong một khối phải được canh lề như nhau

```
if True:
    print "Answer"
    print "True"
else: print "False"
```

- Lệnh được viết trên nhiều dòng sử dụng ký tự \

```
total =    item_one + \  
          item_two + \  
          item_three
```

- Lệnh được bao bằng các cặp dấu ngoặc: [], {}, () không cần phải sử dụng ký tự \ để tiếp tục dòng

```
days = ['Monday', 'Tuesday', 'Wednesday',  
        'Thursday', 'Friday']
```

- Dấu ; để cách nhiều lệnh trên dòng

```
import sys; x = 'foo'; sys.stdout.write(x + '\n')
```

- Nhóm nhiều câu lệnh đơn tạo nên một khối lệnh và cũng được gọi là bộ (suites)
- Các lệnh phức như if, while, def và class cần một dòng header và một bộ
- Dòng header bắt đầu câu lệnh (bằng một từ khoá tương ứng ví dụ như if, def, ...) và kết thúc bằng dấu hai chấm : theo sau là một suite

```
def hi(name):  
    print 'Hello ' + name  
    print 'Have a good day!'  
hi('nghi')
```

Lệnh print trong Python 2

```
>>> print 'hello'
hello
>>> print 'hi','there'
hi there
>>> a = 7.0
>>> b = 2
>>> name = 'toto'
>>> print '%f/%d = %f' %(a, b, a/b)
7.000000/2 = 3.500000
>>> print 'Hi %s' %name
Hi toto
```

Q&A: %f %d %s meaning? [6](#)

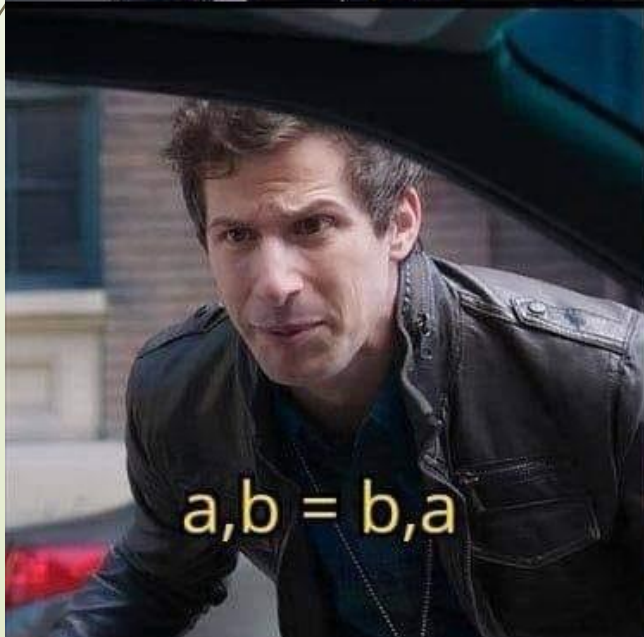
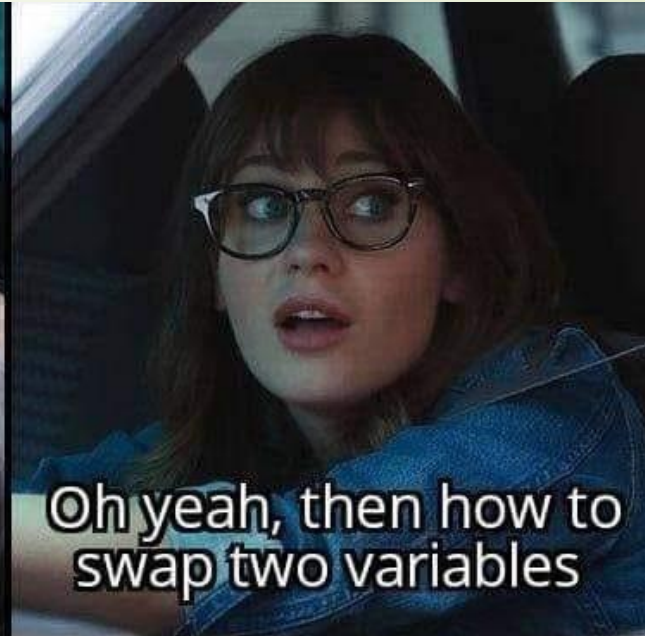
Lệnh Input trong Python

```
>>> name = input("Please enter your name: ")
Please enter your name: 'toto'
>>> print 'Hi %s' %name
Hi toto
>>> a = input("a = ")
a = 5
>>> b = input("b = ")
b = 7.2
>>> a+b
12.2
```

- Tên: ký tự bắt đầu phải là **alphabet** hoặc **_**
- Không cần khai báo, chỉ gán giá trị (sử dụng dấu **=**)
- Được tạo ra trong lần đầu gán giá trị
- Phạm vi biến: cục bộ, toàn cục
- Tham khảo đến đối tượng
- Thông tin về kiểu gắn liền với đối tượng
- Kiểu cơ bản: int, float, complex, bool, string
- Các phép toán số học: +, -, *, /, %, **
- Phép toán so sánh: ==, !=, >, >=, <, <=
- Phép toán luận lý: and, or, not

```
>>> item_name = 'Computer'
>>> item_qty = 10
>>> item_value = 1000.23
>>> print item_name, item_qty, item_value
Computer 10 1000.23
>>> x = y = z = 1
>>> print x, y, z
1 1 1
>>> x,y,z = 1,2,'abcd'
>>> print x, y, z
1 2 abcd
>>> x, y = y, x
>>> print x, y
2 1
```


Biến, kiểu cơ bản, phép toán



```
>>> var1 = "Python"
>>> def func1():
...     var1 = "PHP"
...     print "In side func1() var1 = ",var1
...
>>> def func2():
...     global var1
...     print "In side func2() global var1 = ",var1
...
>>> func1()
In side func1() var1 =  PHP
>>> func2()
In side func2() global var1 =  Python
```

Biến, kiểu cơ bản, phép toán

```
>>> x = 7.2
>>> x
7.2
>>> type(x)
<type 'float'>
>>> x = 'hello'
>>> x
'hello'
>>> type(x)
<type 'str'>
>>> x = 123456789
>>> x ** 2
15241578750190521
>>> z = 3+2j
>>> t = 2-1j
>>> z+t
(5+1j)
```

Biến, kiểu cơ bản, phép toán

```
>>> x = 7
```

```
>>> y = 2
```

```
>>> x%y
```

```
1
```

```
>>> x/y
```

```
3
```

```
>>> 1.0*x/y
```

```
3.5
```

```
>>> float(x)/y
```

```
3.5
```

```
>>> a = 7.2
```

```
>>> int(a)%y
```

```
1
```

```
>>> b = True
```

```
>>> type(b)
```

```
<type 'bool'>
```

Biến, kiểu cơ bản, phép toán

```
>>> s = 'Hello World!'
>>> type(s)
<type 'str'>
>>> len(s)
12
>>> print s
Hello World!
>>> print s[0]
H
>>> print s[-1]
!
>>> print s[2:5]
llo
>>> print s[2:]
llo World!
>>> print s * 2
Hello World!Hello World!
>>> name = 'Ben ' + str(10)
>>> print name
Ben 10
```

Cấu trúc điều khiển

- Cấu trúc rẽ nhánh **if**

```
if (cond1):  
    ...  
elif (cond2):  
    ...  
else:  
    ...
```



```
#eq:  $ax + b = 0$   
a = input('a = ')  
b = input('b = ')  
if (a==0):  
    if (b==0):  
        print 'Pt vo dinh'  
    else:  
        print 'Pt vo nghiem'  
else:  
    x = -1.0*b/a  
    print 'x = ', x
```

- Cấu trúc lặp **while**

```
while (cond):  
    ...  
    loop_body  
    ...
```



```
#s = 1 + 2 + ... + n  
n = input('n = ')  
s = 0  
i = 1  
while (i<=n):  
    s = s + i  
    i = i + 1  
  
print 's = 1 + 2 + ... + %d = %d' %(n, s)
```


- Cấu trúc lặp **for**

```
for iter_var in sequence:
```

```
...
```

```
    loop_body
```

```
...
```



```
#s = 1 + 2 + ... + n
```

```
n = input('n = ')
```

```
s = 0
```

```
for i in range(1, n+1):
```

```
    s = s + i
```

```
print 's = 1 + 2 + ... + %d = %d' % (n, s)
```

- Hàm xây dựng sẵn trong các mô-đun

```
>>> import os
>>> import random as rand
>>> from math import sqrt, cos, sin
>>> os.system('ls -l')
total 32
-rw-rw-r-- 1 nghi nghi 132 Dec 20 19:37 for.py
-rw-rw-r-- 1 nghi nghi  41 Dec 20 11:36 hello.py
-rw-rw-r-- 1 nghi nghi  99 Dec 20 11:39 hi.py
-rw-rw-r-- 1 nghi nghi 729 Dec 20 15:28 var.py
-rw-rw-r-- 1 nghi nghi 141 Dec 20 19:32 while.py
0
>>> r = rand.random()
>>> print r
0.617259044825
>>> x = 64
>>> sqrt(64)
8.0
```

- Hàm do lập trình viên định nghĩa

```
def function_name ([parameters]):  
    ...  
    body_of_the_function  
    ...  
    return ...
```

- Tham số tùy chọn, có thể đặt giá trị mặc định
- Hàm có thể hoặc không trả về kết quả hoặc
- đệ quy

```
>>> def hi():  
...     print 'Hi there!'  
...  
>>> hi()  
Hi there!  
>>>  
>>> def invite(name = 'Toto'):  
...     print 'Dear', name  
...     print 'It will be a great pleasure'  
...     print 'if you attend the birthday party of Tutu'  
...  
>>> invite('Philippe')  
Dear Philippe  
It will be a great pleasure  
if you attend the birthday party of Tutu
```

```
>>> def addTwo(a, b):  
...     return a + b  
...  
>>> def divide(a, b):  
...     return a/b, a%b  
...  
>>> x = 2  
>>> y = 3  
>>> z = addTwo(x, y)  
>>> print z  
5  
>>> p, q = divide(x, y)  
>>> print p, q  
0 2
```

```
>>> def fact(n):  
...     if (n == 1):  
...         return 1  
...     else:  
...         return n*fact(n-1)  
...  
>>> n = 5  
>>> print '%d! = %d' %(n,fact(n))  
5! = 120
```

Các kiểu dữ liệu phức: Kiểu String

```
>>> a = 'Hello world!'
>>> b = "Hello world!"
>>> a == b
True
>>> a = "Khang's lecture"
>>> print a
Khang's lecture
>>> a = "One line.\nAnother line."
>>> print a
One line.
Another line.
>>> b = """One line,
... another line,
... and some..."""
>>> print b
One line,
another line,
and some...
```


Các kiểu dữ liệu phức: Kiểu String

```
>>> a = "58"
>>> type(a)
<type 'str'>
>>> b=int(a)
>>> b
58
>>> type(b)
<type 'int'>
>>> f = float('1.2e-3')
>>> f
0.0012
>>> print f
0.0012
>>> 0.0012
0.0012
>>> eval('23-12')
11
```

Các kiểu dữ liệu phức: Kiểu String

```
>>> a = "Part 1"
>>> b = "and part 2"
>>> a + ' ' + b
'Part 1 and part 2'
>>> s = a * 2
>>> print s
Part 1Part 1
>>> s[0]
'P'
>>> s[0:4]
'Part'
>>> s[5:]
'1Part 1'
>>> s[-1]
'1'
>>> s[6:-1]
'Part '
```

Các kiểu dữ liệu phức: Kiểu String

```
>>> print s
Part 1Part 1
>>> len(s)
12
>>> 'p' in s
False
>>> 'P' in s
True
>>> 'Part' in s
True
>>> s[0] = 'B'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
>>> s = 'B' + s[1:]
>>> print s
Bart 1Part 1
```

Các kiểu dữ liệu phức: Kiểu String

```
>>> s = 'a string, with stuff'
>>> s.count('st')
2
>>> s.find('stu')
15
>>> s.split(' ')
['a', 'string,', 'with', 'stuff']
>>> three = '3'
>>> three.isdigit()
True
>>> supper = s.upper()
>>> supper
'A STRING, WITH STUFF'
>>> s.rjust(30)
'          a string, with stuff'
>>> "newlines\n\n\n".strip()
'newlines'
```

Các kiểu dữ liệu phức: Kiểu List (Ordered Collection of Objects)

```
>>> r
[1, 2.0, 3, 5]
>>> type(r)
<type 'list'>
>>> r[1]
2.0
>>> r[-1]
5
>>> r[1:3]
[2.0, 3]
>>> w = r + [10, 19]
>>> w
[1, 2.0, 3, 5, 10, 19]
>>> t = [0.0] * 10
>>> t
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
```

Các kiểu dữ liệu phức: Kiểu List (Ordered Collection of Objects)

```
>>> r = [1, 2.0, 3, 5]
>>> r[3] = 'word'
>>> r
[1, 2.0, 3, 'word']
>>> r[0] = [9, 8]
>>> r
[[9, 8], 2.0, 3, 'word']
>>> r[0:3] = [1, 2, 5, 6]
>>> r
[1, 2, 5, 6, 'word']
>>> r[1:3] = []
>>> r
[1, 6, 'word']
>>> len(r)
3
>>> 6 in r
True
>>> r.index(6)
1
```

Các kiểu dữ liệu phức: Kiểu List (Ordered Collection of Objects)

```
>>> r = [1, 2.0, 3, 5]
>>> r.append('thing')
>>> r
[1, 2.0, 3, 5, 'thing']
>>> r.append(['another', 'list'])
>>> r
[1, 2.0, 3, 5, 'thing', ['another', 'list']]
>>> r = [1, 2.0, 3, 5]
>>> r.extend(['item', 'another'])
>>> r
[1, 2.0, 3, 5, 'item', 'another']
>>> k = r.pop()
>>> k
'another'
>>> r
[1, 2.0, 3, 5, 'item']
```

Các kiểu dữ liệu phức: Kiểu List (Ordered Collection of Objects)

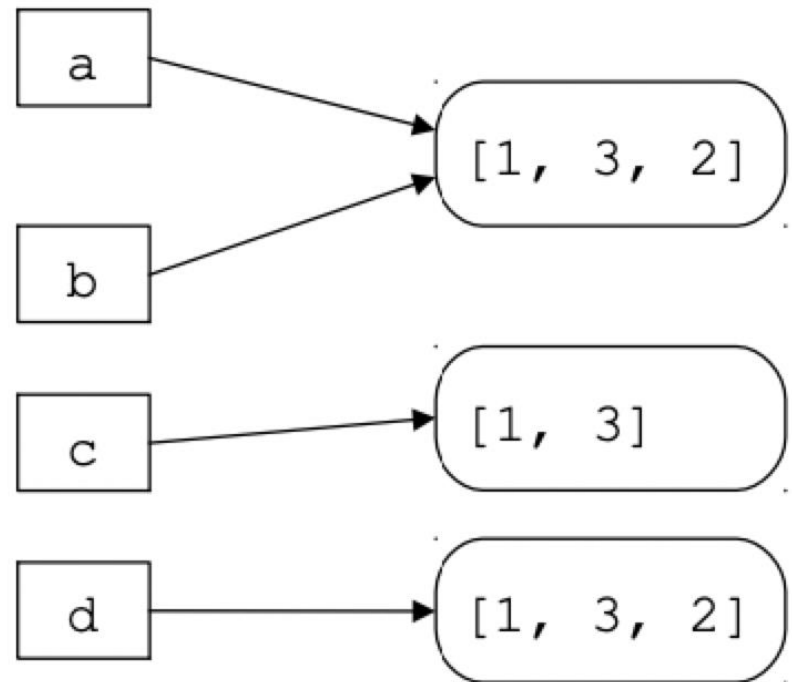
```
>>> r = [2, 5, -1, 0, 20]
>>> r.sort()
>>> r
[-1, 0, 2, 5, 20]
>>> w = ['apa', '1', '2', '1234']
>>> w.sort()
>>> w
['1', '1234', '2', 'apa']
>>> w.reverse()
>>> w
['apa', '2', '1234', '1']
>>> v = w[:]
>>> v.reverse()
>>> v
['1', '1234', '2', 'apa']
>>> w
['apa', '2', '1234', '1']
```


Các kiểu dữ liệu phức: Kiểu List (Ordered Collection of Objects)

```
>>> s = 'biovitrum'
>>> w = list(s)
>>> w
['b', 'i', 'o', 'v', 'i', 't', 'r', 'u', 'm']
>>> w.reverse()
>>> w
['m', 'u', 'r', 't', 'i', 'v', 'o', 'i', 'b']
>>> r = ''.join(w)
>>> r
'murtivoib'
>>> d = '-'.join(w)
>>> d
'm-u-r-t-i-v-o-i-b'
>>> s = 'a few words'
>>> w = s.split()
>>> w
['a', 'few', 'words']
>>> ' | '.join(w)
'a | few | words'
```

Các kiểu dữ liệu phức: Kiểu List (Ordered Collection of Objects)

```
>>> a = [1, 3, 2]
>>> b = a
>>> c = b[0:2]
>>> d = b[:]
>>> b.sort()
>>> a
[1, 2, 3]
```



Các kiểu dữ liệu phức: Kiểu Tuples (as List, except immutable)

```
>>> t = (1, 3, 2)
>>> t[1]
3
>>> (a, b, c) = t
>>> a
1
>>> b
3
>>> a, b, c
(1, 3, 2)
>>> a, b = b, a
>>> a, b
(3, 1)
>>> r = list(t)
>>> r
[1, 3, 2]
>>> tuple(r)
(1, 3, 2)
```

Các kiểu dữ liệu phức: Kiểu Dictionary (An unordered collection of key/value pairs)

```
>>> h = {'key': 12, 'nyckel': 'word'}
>>> h['key']
12
>>> h.has_key('nyckel')
True
>>> h['Per'] = 'Kraulis'
>>> h
{'nyckel': 'word', 'Per': 'Kraulis', 'key': 12}
>>> {'nyckel': 'word', 'Per': 'Kraulis', 'key': 12}
{'nyckel': 'word', 'key': 12, 'Per': 'Kraulis'}
>>> h['Per'] = 'Johansson'
>>> h
{'nyckel': 'word', 'Per': 'Johansson', 'key': 12}
>>> h = {'key': 12, 'nyckel': 'word'}
>>> del h['key']
>>> h
{'nyckel': 'word'}
```

Các kiểu dữ liệu phức: Kiểu Dictionary (An unordered collection of key/value pairs)

```
>>> h = {'key': 12, 'nyckel': 'word'}
>>> 'Per' in h
False
>>> h['Per']
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'Per'
>>> h.get('Per', 'unknown')
'unknown'
>>> h.get('key', 'unknown')
12
>>> h.keys()
['nyckel', 'key']
>>> h.values()
['word', 12]
>>> len(h)
2
```

Các kiểu dữ liệu phức: Kiểu Dictionary (An unordered collection of key/value pairs)

```
>>> h = {'key': 12, 'nyckel': 'word'}
>>> g = h.copy()
>>> del h['key']
>>> h
{'nyckel': 'word'}
>>> g
{'nyckel': 'word', 'key': 12}
>>> h['Per'] = 'Johansson'
>>> h
{'nyckel': 'word', 'Per': 'Johansson'}
>>> h.update(g)
>>> h
{'nyckel': 'word', 'key': 12, 'Per': 'Johansson'}
```

Ask questions!



Think Python, Chapter 1

Exercise 1.2. *Start the Python interpreter and use it as a calculator.*

1. *How many seconds are there in 42 minutes 42 seconds?*
2. *How many miles are there in 10 kilometers? Hint: there are 1.61 kilometers in a mile.*
3. *If you run a 10 kilometer race in 42 minutes 42 seconds, what is your average pace (time per mile in minutes and seconds)? What is your average speed in miles per hour?*

Think Python, Chapter5

Exercise 5.1. *The `time` module provides a function, also named `time`, that returns the current Greenwich Mean Time in “the epoch”, which is an arbitrary time used as a reference point. On UNIX systems, the epoch is 1 January 1970.*

```
>>> import time
>>> time.time()
1437746094.5735958
```

Write a script that reads the current time and converts it to a time of day in hours, minutes, and seconds, plus the number of days since the epoch.

Think Python, Chapter 2

Exercise 2.2. *Practice using the Python interpreter as a calculator:*

1. *The volume of a sphere with radius r is $\frac{4}{3}\pi r^3$. What is the volume of a sphere with radius 5?*
2. *Suppose the cover price of a book is \$24.95, but bookstores get a 40% discount. Shipping costs \$3 for the first copy and 75 cents for each additional copy. What is the total wholesale cost for 60 copies?*
3. *If I leave my house at 6:52 am and run 1 mile at an easy pace (8:15 per mile), then 3 miles at tempo (7:12 per mile) and 1 mile at easy pace again, what time do I get home for breakfast?*

Think Python, Chapter3

Exercise 3.3. *Note: This exercise should be done using only the statements and other features we have learned so far.*

1. Write a function that draws a grid like the following:

```
+ - - - - + - - - - +
|           |           |
|           |           |
|           |           |
|           |           |
+ - - - - + - - - - +
|           |           |
|           |           |
|           |           |
|           |           |
+ - - - - + - - - - +
```

Hint: to print more than one value on a line, you can print a comma-separated sequence of values:

```
print('+', '-')
```

By default, print advances to the next line, but you can override that behavior and put a space at the end, like this:

```
print('+', end=' ')
print('-')
```

The output of these statements is '+ -' on the same line. The output from the next print statement would begin on the next line.

2. Write a function that draws a similar grid with four rows and four columns.